Supply Chains as Directed Containers

Robin Langer Claude

July 2026

Abstract

We describe supply chains using the language of containers and directed containers, climbing a ladder of examples while the formalism stands still. The shipping network of *Sawasdee Cellars*, an Australian wine exporter selling into Thailand, is a finite tree, and its free category is a directed container (and therefore a comonad); cold-chain integrity, tariffs, markups, and licensing compliance are monoid-valued tracking functors on it. *Falcon Steel*, a producer in the United Arab Emirates, adds a second tree—the bill of materials—as a second directed container, with sourcing a functor connecting it to the network. Alternative routings then make the network a DAG, and recycling closes a cycle, making it a graph in earnest; at every rung the constructions apply unchanged. Aggregation and demand fulfilment are cofunctors, and inventory dynamics form an update monad. The distinction between the flow of goods (functors) and the flow of demand (cofunctors) is genuine and irreducible; following Fairbanks–Carlson–Spivak, the two coexist in a double category. This is an expository note—no new theorems, just a translation exercise to build intuition. All constructions are mechanically verified in Lean 4.

1 Introduction

Supply chain management is a discipline of local records that must cohere globally: temperatures logged per leg but certified per journey, margins taken per hand-off but audited in the shelf price, parts lists written per product but costed per finished good. The mathematics of local views that cohere is the theory of comonads, and the comonads relevant here have a concrete combinatorial form: *directed containers*, equivalently small categories, equivalently comonoids in **Poly**. This note translates the standard vocabulary of supply chains into that language, one term at a time.

The note is organized as a ladder of examples on which the formalism never changes—only the underlying graph does. Section 2 fixes the running example, Sawasdee Cellars, a Melbourne agency exporting South Australian shiraz to Thailand; Section 3 is a glossary of supply-chain terms, aimed at a reader who knows what a comonad is but not what a duty point is; Section 4 recalls containers and directed containers. Then the ladder. *Trees*: the wine network is a tree, and cold chain, tariffs, markups, and licensing are monoid-valued tracking functors on its free category (Section 5); Falcon Steel, a producer in the United Arab Emirates, adds a second tree, the bill of materials, with sourcing a functor connecting the two (Section 6). *DAGs*: alternative routings give the network several routes between the same endpoints, and repeated components do the same for the BOM (Section 7). *Graphs*: the mill's scrap loop closes a cycle, and the free category takes it in stride (Section 8). With the flow of goods in place, Section 9 turns to the flow of demand: aggregation and fulfilment are cofunctors, which coexist with functors in a double category but merge with them in neither direction. Section 10 models inventory as an update monad. Section 11 records the Lean 4 formalization of every construction in the note, and Section 12 points at dynamics.

2 Setup: Simplified Supply Chain

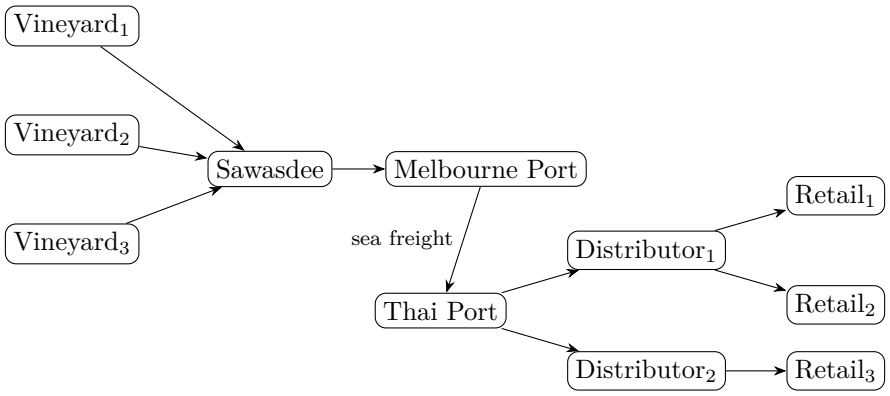
We work with a stripped-down model. A supply chain has:

- A set Fac of **facilities**: vineyards, export agencies, ports, distributors, retailers.
- A set of **shipping legs**: a leg $f \to g$ means goods can move directly from facility f to facility g. Facilities and legs form a directed graph G.
- **Per-leg tracking data**: each shipping leg carries the duties and taxes it triggers, the markup taken at its hand-off, the temperatures the goods experience, and the licences it requires. Section 5.1 makes these compositional.
- **Inventory**: facilities may hold stock, per product, modified over time by **receipts** (goods arriving) and **issues** (goods dispatched).

Our running example is *Sawasdee Cellars*, a Melbourne export agency. Sawasdee buys finished, bottled shiraz directly from three vineyards and sells it into Thailand; it does no bottling, holds no stock, and runs no warehouse. What passes through Sawasdee is *title*, not pallets: it takes ownership, arranges the freight, and adds its markup. It is nonetheless a genuine node of the network—goods change custody and price there, even though they never sit on a Sawasdee shelf:

Facility	Location	Role
Vineyard_1	Barossa Valley	grows and bottles wine
Vineyard_2	Clare Valley	grows and bottles wine
Vineyard_3	McLaren Vale	grows and bottles wine
Sawasdee	Melbourne	export agency
Melbourne Port	Melbourne	container port
Thai Port	Chonburi	container port
Distributor_1	Bangkok	distributor
Distributor_2	Chiang Mai	distributor
Retail_1	Bangkok	retail wine shop
Retail_2	Bangkok	retail wine shop
Retail_3	Chiang Mai	retail wine shop

The shipping graph:



The model above is everything a pure *distributor* needs: nothing is manufactured, everything is bought finished, and goods keep their identity from vineyard to shelf. Notice also that the shipping graph is a *tree*: the first rung of the ladder. Sections 5 and 6 stay on that rung; Sections 7 and 8 climb.

3 Glossary: Supply-Chain Terms

Supply chain management has its own vocabulary, standardized by decades of industry practice. This section collects the terms used in this note, aimed at a reader who knows what a comonad is but not what a duty point is.

Markup / margin. The percentage a tier adds when goods change hands: the difference between what it pays and what it charges. Margins compound multiplicatively along a route—each hand-off applies $\times(1+m)$—and enter the fiscal stack at a definite point (Section 5.1).

Landed cost. The full cost of goods delivered at a destination: purchase price plus freight, insurance, duties, and taxes. In Section 5.1 it is the value of a composite affine map applied to the ex-cellar price.

Sourcing. The assignment of a producer to each thing that must be made or bought: *which facility makes which component*. Section 6.2 makes this a functor.

Facility. Any node of the network through which goods pass: vineyard, port, distributor, retailer. A facility need not be a physical place that holds stock—Sawasdee itself is an agency, a node where goods change *custody and price* but never sit on a shelf. Legal nodes and physical nodes enter the network on equal terms (compare the bonded warehouse, below).

Shipping leg / route. A leg is a single direct movement capability between two facilities; a route is a composable sequence of legs. Legs generate, routes compose: Section 5 takes the free category.

Distributor / distribution centre. A facility that holds finished goods close to customers and ships only downstream—toward retail, never back into production (Distributor_1 in Bangkok and Distributor_2 in Chiang Mai, in our example).

Last mile. The final delivery leg, from the last stocking point to the customer or retailer. The shortest legs of the network and collectively the most expensive—typically 40–50% of total logistics cost—because the trunk network's consolidation fans out into many small terminal shipments. For wine it must also be the last link of the cold chain (Section 5.1).

Cold chain. The requirement that a temperature-sensitive product stay within a temperature band along its *entire* journey—refrigerated warehouses, insulated or actively cooled ("reefer") containers, and controlled handoffs between them. One breach anywhere spoils the whole chain (Section 5.1).

Tariff / customs duty. A tax levied when goods cross a border. A **specific** duty is a fixed amount per unit; an **ad valorem** duty is a percentage of declared value. They accumulate along a route additively and multiplicatively respectively (Section 5.1).

Excise; duty point. A tax on certain goods (alcohol among them) triggered not by movement but by an event: the **duty point**, typically release into home consumption. Until then goods can move and be stored *in bond*, duty-unpaid (Remark 5.6).

Bonded warehouse. A warehouse authorized by customs to hold goods with duty suspended. A facility distinguished by legal status, not physical function—the same building, bonded or duty-paid, is two different nodes for planning purposes.

Consolidation. The gathering of goods from several origins into one shipment rather than shipping each separately: bottles from three vineyards meet in one export container. In Section 6.2 this is the functoriality of sourcing.

Inventory. The stock currently held at a facility, per product: a vector in $\mathbb{N}^{\text{Prod}}$, where Prod is the set of distinct products handled. Modified by **receipts** (arrivals), **issues** (consumption or dispatch), and **transfers** (movements between facilities)—collectively, *stock movements* (Section 10).

Replenishment policy. A rule that observes current stock and decides what to reorder, when, and how much. Section 10 identifies such policies with Kleisli maps of an update monad.

Lead time. The total delay between deciding to order something and having it available: for an order placed on a vineyard, the time until the case is on the retail shelf. Along a fixed route it is the sum of the transit times of the legs—a tracking functor like any other (Section 5.1).

Aggregate planning. Planning done at a coarsened level—regions instead of facilities, prod-

uct families instead of individual products, quarters instead of days—because demand is forecast and decisions are made coarse before they are executed fine. Section 9 models the coarse-to-fine passage as a cofunctor.

Demand fulfilment. The process of turning expressed demand ("Thailand needs shiraz") into concrete execution (specific facilities shipping along specific routes): the lifting problem of Section 9.

Order book. The ledger of outstanding orders not yet fulfilled: committed future stock movements, as opposed to inventory, which records their accumulated past effect (Section 10).

Bullwhip effect. Forrester's observation that the variance of orders *amplifies* as one moves upstream, away from final demand: retailers see ripples, raw-material suppliers see waves. Each tier reacts to its neighbour's orders rather than to true demand, compounding delay and overcorrection (Section 12).

4 Containers and Directed Containers

We briefly recall the definitions [1, 3].

Definition 4.1. A **container** $S \triangleleft P$ consists of a set S of **shapes** and, for each shape s, a set $P(s)$ of **positions**. Its **extension** is the functor $[\![S \triangleleft P]\!] : \mathbf{Set} \to \mathbf{Set}$ given by

$$[\![S \triangleleft P]\!](X) \;=\; \sum_{s \in S} X^{P(s)},$$

i.e. a shape together with an X-value at every position.

Extensions of containers are exactly the polynomial functors in one variable; in Spivak's notation [11, 10] one writes $p = \sum_{s \in S} y^{P(s)}$ and works in the category **Poly**.

Definition 4.2. A **directed container** [3] is a container $S \triangleleft P$ equipped with:

- a **subshape** operation: for $s \in S$ and $p \in P(s)$, a shape $s \downarrow p \in S$;
- a **root** position $\mathsf{o}_s \in P(s)$ for each s;
- a **translation** operation: for $p \in P(s)$ and $p' \in P(s \downarrow p)$, a position $p \oplus p' \in P(s)$;

subject to five laws: $s \downarrow \mathsf{o}_s = s$; $\quad s \downarrow (p \oplus p') = (s \downarrow p) \downarrow p'$; $\quad p \oplus \mathsf{o} = p$; $\quad \mathsf{o} \oplus p = p$; $\quad (p \oplus p') \oplus p'' = p \oplus (p' \oplus p'')$.

Two theorems make this structure important.

Theorem 4.3 (Ahman–Chapman–Uustalu [3])**.** *Directed container structures on $S \triangleleft P$ are exactly comonad structures on $[\![S \triangleleft P]\!]$. The counit and comultiplication are*

$$\varepsilon(s, v) = v(\mathsf{o}_s), \qquad \delta(s, v) = \big(s,\ \lambda p.\, (s \downarrow p,\ \lambda p'.\, v(p \oplus p'))\big).$$

Theorem 4.4 (Ahman–Uustalu [4]; Spivak et al. [10])**.** *Directed containers are exactly small categories: shapes are objects, $P(s)$ is the set of morphisms out of s, the subshape $s \downarrow p$ is the codomain of p, the root o_s is the identity, and $\oplus$ is composition. Equivalently, directed containers are the comonoids in $(\mathbf{Poly}, \triangleleft, y)$.*

Slogan 4.5. A directed container is a system of *local views*: every shape knows its positions, every position leads to a subshape with its own positions, and the two levels of description agree. The comonad laws *are* this agreement.

Supply chains are full of local views—a warehouse's view of its downstream network, a subassembly's view of its own components—and this is why the language fits.

5 Trees I: The Shipping Network as a Directed Container

We first give the pure mathematics, with no supply chain in sight, and only then interpret it.

Construction 5.1. Let G be a finite directed graph: a set V of nodes and a set of edges, each with a source and a target in V. The nodes are *labelled* in the combinatorialist's sense—V is a set of distinguishable elements, not an isomorphism class—and no node is distinguished as a root. Define the directed container $\mathcal{N} = \mathcal{N}_G$:

- **shapes**: nodes, $S = V$;
- **positions**: $P(v)$ is the set of finite directed paths in G starting at v, including the empty path;
- $v \downarrow \rho$ is the node where the path ρ ends;
- o_v is the empty path at v;
- $\rho \oplus \rho'$ is concatenation: follow ρ, then continue along ρ'.

The five laws are immediate. As a category, $\mathcal{N}$ is the free category on G. In the formalization, the construction is `Digraph.network` in `Network.lean`; each concrete graph G produces its own directed container $\mathcal{N}_G$.

The construction accepts *any* directed graph G. This section works out what happens on the simplest kind.

Proposition 5.2. *If the underlying undirected graph of G is a tree (such a G is called a polytree), then $\mathcal{N}$ is thin: between any two nodes there is at most one path. Thus $\mathcal{N}$ is a poset, the reachability order on nodes, and a position carries no more information than its endpoint:*

$$P(v) \;\cong\; \{\textit{nodes reachable from } v\}.$$

Proof. Two distinct paths from v to the same node would trace out a cycle in the underlying undirected graph. So $\rho \mapsto v \downarrow \rho$ is injective, and it is surjective onto the reachable nodes by definition. □

$\mathcal{N}$ is exactly the directed container of a finite tree: shapes are the nodes, and each node's positions are the nodes downstream of it (itself included).

This pure structure maps directly onto the supply chain of Section 2, with nothing added and nothing forced:

Pure structure	Supply-chain reading
node of G	facility
edge of G	shipping leg
directed path	shipping route
empty path o_f	goods staying at f
endpoint $f \downarrow \rho$	destination of the route
concatenation $\oplus$	composition of routes
polytree hypothesis	the wine network's shape

For Sawasdee Cellars: $P(\text{Vineyard}_1)$ has nine elements—the empty route, and one route for each of the eight facilities reachable from Vineyard_1: through Sawasdee and the Melbourne Port, across the sea to the Thai Port, branching to the two distributors, and fanning out to the three retailers. $P(\text{Retail}_1)$ has only the empty route: retail is the end of the line. Two kinds of leg deserve comment. The **deep-sea leg** Melbourne Port → Thai Port is the longest: ten days in a refrigerated ("reefer") container. The final legs from the distributors to the retailers—Distributor_1 to Retail_1 and Retail_2, Distributor_2 to Retail_3—are the **last mile**: refrigerated van deliveries, the shortest legs of the network and the only ones the customer ever sees. They

are legs like any other—but each is the last link of the cold chain, and by idempotence of max (Section 5.1) a single warm van undoes the refrigerated integrity of the entire route from the vineyard. In Bangkok, at 35°C ambient, emphatically so.

Note that the tree has no marked root—and could not be given one. In a rooted tree the root does real work: it orients every edge away from itself, so each node has at most one leg in. Here Sawasdee, with three legs in and one out, is not a root but the *waist* of the network, through which every bottle's title passes; no choice of root generates this orientation, which comes from the flow of goods instead. Nor does the theory miss a root: a directed container has a *local* root o_f at every shape—each facility is the root of its own downstream view. Rooted trees will appear in Section 6, used in the opposite way: there each entire rooted tree is a single *shape*, and the root is essential data—the product being made. Distribution has no privileged vantage point; manufacture has a privileged product.

An element of $[\![\mathcal{N}]\!](X)$ is a facility f together with an X-value on every route out of f: a **decorated downstream view**. Take $X = \mathbb{R}$ and decorate each route with its transit time; or take $X = \{\text{ok}, \text{delayed}, \text{blocked}\}$ and decorate each route with its status.

The comonad structure is where the interpretation earns its keep:

- The counit ε reads the value at the empty route: *look at your own facility.*
- The comultiplication δ sends a decorated view at f to a view-of-views: each route ρ out of f is decorated with the *entire downstream view of the facility at the end of* ρ, obtained by reading f's original decoration along composite routes $\rho \oplus \rho'$.

Slogan 5.3. The comonad laws say that Sawasdee's view of the Thai Port, as computed from within Vineyard$_1$'s view, equals the Thai Port's own view. Supply chain visibility is comonadic coherence.

5.1 Tracking along routes: cold chain, tariffs, markups, compliance

Wine export lives under constant surveillance: temperature must stay in range, duties and taxes accrue at borders, every intermediary adds its margin, every leg needs the right licences. All of these are instances of one construction.

Construction 5.4. Let $(M, \oplus, e)$ be a monoid, regarded as a one-object category $\mathbf{B}M$. A **compositional tracking** of M-data is a functor $d : \mathcal{N} \to \mathbf{B}M$: an assignment of $d(\rho) \in M$ to every route, satisfying

$$d(\mathsf{o}_f) = e, \qquad d(\rho \oplus \rho') = d(\rho) \oplus d(\rho').$$

Since $\mathcal{N}$ is free on the shipping graph G—every route is a composite of legs, and distinct sequences of legs are distinct routes, with no equations between them—such a functor is *exactly* a labelling of individual legs: measure per leg, and the functor computes per route. Audit locally, certify globally—that is the universal property of the free category.

On a tree (Proposition 5.2) this is especially concrete: a tracking is an edge-labelling of the tree, composed along the unique path between any two facilities.

The choice of monoid is the choice of what is being tracked:

Tracked quantity	Monoid	Character
freight cost, specific duty	$(\mathbb{R}_{\geq 0}, +, 0)$	extensive
ad valorem tariff, markup	$(\mathbb{R}_{>0}, \times, 1)$	multiplicative
peak temperature	$(\mathbb{R} \cup \{-\infty\}, \max, -\infty)$	path-critical
cold chain intact?	$(\{\text{ok}, \text{broken}\}, \wedge, \text{ok})$	idempotent
licences required	$(\mathcal{P}(L), \cup, \emptyset)$	accumulative
price build-up	$(\text{Aff}, \circ, \text{id})$	order-sensitive

- **Taxes and tariffs.** Specific duties (baht per case) add along a route; ad valorem duties (a percentage of value) compose multiplicatively. Both are functors; only the target monoid differs. A leg that crosses no border is labelled e.
- **Markups.** Wine changes hands at each tier: vineyard to Sawasdee, Sawasdee to distributor, distributor to retailer. Each hand-off applies a margin $\times(1+m)$; label each leg with the margin taken at its source, and the functor into $(\mathbb{R}_{>0}, \times, 1)$ compounds them. The gap between ex-cellar price and Bangkok shelf price is a composite morphism.
- **Cold chain.** Peak temperature composes by max—an *idempotent* monoid, which is precisely the formal content of "chain": a single hot hour on the wharf dominates ten cold days at sea, and no subsequent refrigeration composes it away. The composite is as good as its worst morphism.
- **Licences and regulatory compliance.** Let L be the set of permits: export licences, import licences, alcohol distribution licences, organic and appellation certifications. The functor $d : \mathcal{N} \to \mathbf{B}(\mathcal{P}(L))$ assigns each route the set of licences it requires—the union over its legs. An operator holding licence set $A \subseteq L$ may lawfully use route ρ iff $d(\rho) \subseteq A$. Compliance checking is a subset test against a functorially aggregated requirement.
- **Price build-up: landed cost and shelf price.** The full pricing rule mixes multiplicative pieces (margins, ad valorem duty, VAT) with additive ones (freight per case, specific duty, excise). Neither monoid alone suffices. The natural target is the monoid Aff of affine maps $v \mapsto av + b$ ($a > 0$, $b \geq 0$) under composition: label each leg with the affine map its hand-off applies, and the composite $d(\rho) \in \mathrm{Aff}$ turns an ex-cellar price into a price at the end of the route, $v \mapsto d(\rho)(v)$.

Remark 5.5. Aff is *not* commutative, and this is a feature, not a defect: a 10% margin applied before a specific excise of b baht differs from the same margin applied after it, by exactly $b/10$. Statutory stacking rules are order-sensitive—Thai customs duty is assessed on the CIF value, excise on top of that, VAT on the duty- and excise-inclusive value, local taxes on top again, and each tier's margin enters at its own point in the chain. Legs of a route occur in a definite order; the free category remembers that order; and a functor $d : \mathcal{N} \to \mathbf{B}\mathrm{Aff}$ maps the order of hand-offs to the order of function composition. Commutative trackings are the special case where the bookkeeping may be done in any order—the fiscal ones may not. (On the Australian side the stack is short: exports are free of GST and of WET, the wine equalisation tax, so the legs up to the Melbourne Port carry only freight and margin.)

Each tracking functor generates decorated views: the assignment $f \mapsto (\rho \mapsto d(\rho))$ is an element of $[\![\mathcal{N}]\!](M)$ over every shape at once, and Slogan 5.3 applies verbatim: $\mathrm{Vineyard}_1$'s record of duties and temperatures beyond Sawasdee agrees with Sawasdee's own record. Traceability—the auditable handoff between successive custodians that customs and insurers demand—is comonadic coherence.

Remark 5.6. One compliance notion resists this pattern: *excise*. Excise duty is not accrued along a route but triggered by an event—release from bond, the *duty point*. A bonded warehouse and a duty-paid warehouse may be the same building in different legal states. Tracking state changed by events is the update-monad side of the theory, taken up in Section 10; the duty point is a stock movement in a fiscal, not physical, dimension.

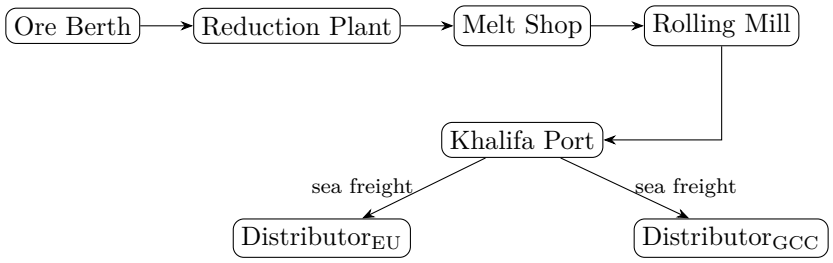
6 Trees II: Bills of Materials—Steel

Everything so far ran on a single directed container, the network $\mathcal{N}$, and that was enough for a reason worth stating: Sawasdee Cellars is a pure *distributor*. A bottle leaving $\mathrm{Vineyard}_1$ is the same bottle on the Bangkok shelf; the network moves goods but never transforms them, so one ontology—facilities and routes—suffices, and every compliance question is route-borne. To see what manufacturing adds, we change examples: *Falcon Steel*, a producer in Abu Dhabi's Mussafah industrial area, making reinforcing bar (rebar) for export.

The pipeline is gas-based, as is standard in the Gulf. Imported **iron ore pellets** arrive at the ore berth and are reduced to **DRI** (direct reduced iron) at a reduction plant; the melt shop melts DRI together with **scrap** in an electric arc furnace and casts **billets**; the rolling mill rolls billets into **rebar**, which leaves through Khalifa Port for two overseas distributors—one in Rotterdam serving EU construction, one in Dammam serving the Gulf:

Facility	Location	Role
Ore Berth	Mussafah	receives imported pellets
Reduction Plant	Mussafah	reduces pellets to DRI (gas)
Melt Shop	Mussafah	EAF; melts DRI + scrap, casts billets
Rolling Mill	Mussafah	rolls billets into rebar
Khalifa Port	Abu Dhabi	container port
$\text{Distributor}_{\text{EU}}$	Rotterdam	EU distributor
$\text{Distributor}_{\text{GCC}}$	Dammam	Gulf distributor

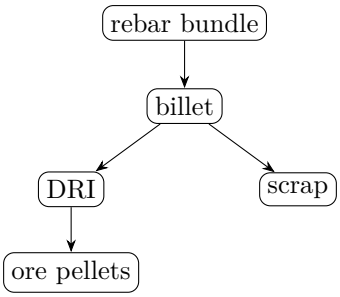
The shipping graph:



This graph is a directed container exactly as in Construction 5.1: call it $\mathcal{N}_{\text{steel}}$, a second instance of the *same* construction on a different graph (the wine network $\mathcal{N}$ was the first; in the formalization both are the one definition `network` of `Network.lean` applied to two concrete graphs, with `Steel.graph` in `Sourcing.lean` covering the four production facilities). The underlying undirected graph is again a tree—a spine with one fan-out at the port—so Proposition 5.2 applies: $\mathcal{N}_{\text{steel}}$ is thin, and $P(\text{Ore Berth})$ has seven elements, the empty route plus one route to each facility downstream; $P(\text{Distributor}_{\text{EU}})$ has one. Every tracking functor of Section 5.1 applies verbatim, though the monoids that carry weight have changed with the industry:

- **Duties.** The EU leg attracts a safeguard or anti-dumping duty, ad valorem: $(\mathbb{R}_{>0}, \times, 1)$. Freight and specific duties add in $(\mathbb{R}_{\geq 0}, +, 0)$.
- **Certificates.** A rebar consignment travels with its **mill test certificate** (EN 10204 type 3.1, attesting chemistry and mechanical properties) and its certificate of origin: $(\mathcal{P}(L), \cup, \emptyset)$, exactly the licence functor of the wine network with a different L.
- **What is absent.** Steel needs no cold chain; the idempotent monoid sits unused. Its path-critical role reappears as $(\max, +)$ lead time in Section 6.3.

A path with a fan-out is a tree, so we have not yet left the first rung. What *is* new is that goods now change identity: pellets are consumed and DRI appears; the product on a facility's outbound leg is not the product on its inbound leg. The document recording these identity changes is the **bill of materials** (BOM): the recursive parts list of a manufactured product—the direct components needed to make one unit, each component carrying its own BOM. Manufactured components (the billet) are called **subassemblies**; raw materials and purchased goods (ore pellets, scrap) are the leaves. Purchasing, costing, and scheduling all read from this document:



6.1 Bills of materials as a directed container

Construction 6.1. A **BOM tree** is a finite rooted tree whose nodes are labelled by products: the children of a node are the direct components of one unit of it. Define the directed container $\mathcal{B}$:

- **shapes**: BOM trees;
- **positions**: $P(t)$ is the set of **occurrences** in t—descending paths from the root, including the empty path;
- $t \downarrow p$ is the subtree at p: the occurrence's own BOM;
- o_t is the empty occurrence: the product itself;
- $p \oplus p'$ is path concatenation: an occurrence inside a component, seen from the finished product.

The five laws are immediate: $\mathcal{B}$ is the directed container of rooted trees, with positions the paths—a second directed container, living beside—not on—the network; its shapes are documents, not facilities. In the formalization it is `bom` in `Sourcing.lean` (a separate construction from `Digraph.network`; a different directed container, not another application of $\mathcal{N}_{(-)}$), whose shapes additionally carry the routing choices needed by Construction 6.3 below (see Section 11).

Concretely, Falcon Steel's rebar BOM drawn above has five occurrences:

$$\langle\rangle, \quad \text{billet}, \quad \text{billet} \cdot \text{DRI}, \quad \text{billet} \cdot \text{scrap}, \quad \text{billet} \cdot \text{DRI} \cdot \text{ore},$$

the empty occurrence being the rebar itself. Note the shift of ontology between the two containers: in $\mathcal{N}_{\text{steel}}$ a position at a shape is a route *out of a facility*; in $\mathcal{B}$ it is a place *inside a document*. And where Section 5 argued that the shipping tree has no root and could not be given one, here the root is essential data: a BOM tree without its root product is not a parts list of anything.

The comonad structure translates industry practice directly. The counit reads the value at the empty occurrence: *look at the product itself*. The comultiplication decorates every occurrence with the entire sub-BOM below it—which is precisely what material requirements planning (MRP) software calls **BOM explosion**.

Slogan 6.2. BOM explosion is comultiplication. The comonad law $s \downarrow (p \oplus p') = (s \downarrow p) \downarrow p'$ says the billet's parts list, read off from within the rebar's exploded view, equals the billet's own parts list—multi-level costing is coherent.

An element of $[\![\mathcal{B}]\!](X)$ is a BOM with an X-value at every occurrence: unit cost, weight, production time, embedded carbon.

6.2 Sourcing as a functor

Each product is made or received somewhere:

Product	Facility	
ore pellets	Ore Berth	received at the berth
DRI	Reduction Plant	made
scrap	Melt Shop	bought at the gate
billet	Melt Shop	made
rebar	Rolling Mill	made

Write $\mathrm{site}(t)$ for the facility producing the root product of the tree t.

Construction 6.3. Sourcing extends to a functor $\mathrm{site} : \mathcal{B} \to \mathcal{N}_{\mathrm{steel}}^{\mathrm{op}}$:

- on shapes, $t \mapsto \mathrm{site}(t)$;
- an occurrence $p \in P(t)$ is sent to a chosen **inbound route** in $\mathcal{N}_{\mathrm{steel}}$ from $\mathrm{site}(t \downarrow p)$ to $\mathrm{site}(t)$: the route the component travels to its point of use.

The $(-)^{\mathrm{op}}$ is forced by orientation: positions of $\mathcal{B}$ point *down* the parts tree, from product to component, while goods flow *up* it, from component to product—an occurrence out of t becomes a route *into* $\mathrm{site}(t)$. Functoriality says the empty occurrence is sourced by the empty route, and the route for a nested occurrence is the composite through the intermediate producer: ore reaches the rolling mill only via the reduction plant and the melt shop, changing identity at each stop. **Consolidation**—components gather where their parent is made—is exactly the functor laws.

Worked through for Falcon Steel's deepest occurrence: the chosen inbound leg for each BOM edge is the single shipping leg from the component's producer to its parent's producer, and the ore occurrence $\mathrm{billet} \cdot \mathrm{DRI} \cdot \mathrm{ore}$ factors as $(\mathrm{billet} \cdot \mathrm{DRI}) \oplus \mathrm{ore}$, so functoriality forces

$$\mathrm{site}(\mathrm{billet} \cdot \mathrm{DRI} \cdot \mathrm{ore}) = \underbrace{(\mathrm{Berth} \to \mathrm{Reduction})}_{\text{ore, within DRI's BOM}} \oplus \underbrace{(\mathrm{Reduction} \to \mathrm{Melt} \to \mathrm{Mill})}_{\text{DRI, within rebar's BOM}},$$

the three-leg journey of the ore, read off from the parts list: the component's own journey first, then its subassembly's. This is the consolidation equation checked by `rfl` in the formalization.

In the formalization, the two containers of this section and the functor between them are all in `Sourcing.lean`: $\mathcal{B}$ is `bom`, $\mathcal{N}_{\mathrm{steel}}^{\mathrm{op}}$ is `network graph.op` (reversing the graph rather than opposing the category), and site is `sourcing`, with `DContFunctor.lean` supplying the notion of covariant functor of directed containers. The rebar tree drawn above, ore's three-leg inbound route to the rolling mill, and one consolidation equation are all checked definitionally (`rfl`).

6.3 Roll-ups, and compliance on two containers

A **roll-up** aggregates per-component data to the product level. For data valued in a commutative monoid—unit costs in $(\mathbb{R}_{\geq 0}, +, 0)$, weights likewise—the roll-up is the sum over all occurrences of the BOM tree. Lead time is subtler: over each root-to-leaf path one *adds* production and transit times, then takes the *maximum* over paths—the critical path, living in the $(\max, +)$ semiring rather than a single monoid. The general theory of such aggregations is worked out by Spivak, Garner, and Fairbanks [12].

For Falcon Steel the cost roll-up is a five-line sum: decorate each occurrence of the rebar BOM with the cost *added* there—the element of $[\![\mathcal{B}]\!](\mathbb{R}_{\geq 0})$ a standard costing run produces—and sum over occurrences (illustrative figures, per tonne of rebar):

Occurrence	Cost added	\$/t
$\langle\rangle$ (rebar)	rolling conversion	70
billet	melt + cast conversion	90
billet·DRI	reduction conversion	60
billet·scrap	scrap purchase	80
billet·DRI·ore	pellet purchase	140
roll-up	sum over $P(t)$	440

The *rolled-up cost of the billet alone* is the same sum over the sub-BOM $t\downarrow$ billet—and the coherence law of Slogan 5.3's BOM analogue (Section 6.1) says computing it inside the rebar's exploded view or from the billet's own parts list must agree: $90 + 60 + 80 + 140 = 370$ either way. Multi-level costing is coassociativity, with arithmetic.

The most instructive compliance case lives on *both* containers at once. The EU's carbon border adjustment mechanism (CBAM) requires, at the border, one number: total embedded CO_2 per tonne of rebar. That number is assembled from a decoration of $\mathcal{B}$ (the emissions of each production step, at each occurrence) plus a tracking on $\mathcal{N}_{\text{steel}}$ (the emissions of each shipping leg, as in Construction 5.4), the two glued along site: each occurrence contributes its process emissions and the emissions of its inbound route (illustrative figures, per tonne of rebar):

Occurrence	Process step	t CO_2/t	Inbound route (site)	t CO_2/t
$\langle\rangle$ (rebar)	rolling	0.10	—	—
billet	EAF melt + cast	0.35	Melt → Mill	0.01
billet·DRI	gas reduction	0.70	Reduction → Melt → Mill	0.02
billet·scrap	gate processing	0.02	Melt → Mill	0.01
billet·DRI·ore	pelletizing (upstream)	0.15	Berth → ⋯ → Mill	0.03
total		1.32		0.07

Roughly 1.4 t CO_2 per tonne of rebar—a single auditable figure, computed from two directed containers and the functor connecting them, and the reason the computation is commercially live: the gas-DRI route sits well below the blast-furnace route's ≈ 2.2, and CBAM prices exactly that difference. One boundary subtlety is also structural: the deep-sea leg to Rotterdam is tracked in the same monoid but sits outside CBAM's system boundary—the regulation selects a *sub-tracking*, not a new construction.

Remark 6.4 (What the second example buys). The dividing line between the two examples is now sharp. As long as goods keep their identity along every morphism—distribution—one directed container suffices and all compliance is route-borne. The moment goods merge, split, or change identity—manufacture—a second directed container is forced, together with a functor connecting the two ontologies. When aggregation and demand enter (Section 9), sourcing will supply a second family of compatibility squares alongside the tracking ones.

7 DAGs: Alternative Routes and Repeated Components

Neither Construction 5.1 nor Construction 6.1 used the tree property; only Proposition 5.2 did. So let the graph G be merely directed and *acyclic*. The free category on G still has every $P(v)$ finite and no nontrivial endomorphisms, but it is no longer thin: two nodes may now be joined by several paths, and a position carries strictly more information than its endpoint. That is the entire content of this rung, and it is the pure statement of a supply-chain fact: positions are *routes*, not destinations. Every construction above applies unchanged.

Both examples climb to this rung—each producing a new directed container $\mathcal{N}_{G'}$ from a new graph G'—each for its own reason.

Wine: alternative routings. Add one alternative routing—transshipment via Singapore alongside the direct sailing—and the new directed container $\mathcal{N}_{G'}$ (in the formalization, `Digraph.network G'` in `Rungs.lean`) has two routes from the Melbourne Port to the Thai Port with the same endpoints but different peak temperature, freight, and duty. It is precisely the tracking functors of Section 5.1 that force the two positions apart: a functor out of $\mathcal{N}$ assigns its data per *route*, and the direct sailing and the transshipment disagree in every column of the monoid table.

Steel: repeated components. A billet needing two charges of DRI has two DRI-edges in its BOM tree: two distinct positions of $\mathcal{B}$ with the same subshape—the same product-labelled tree—two occurrences of the same component, which may have different suppliers, different mill certificates, different embedded carbon. This can never happen in the network, whose nodes are facilities and occur once each; it happens in the BOM because its nodes are merely *labelled* by products, and labels may repeat. So $\mathcal{B}$ fails to be thin for a manufacturing reason rather than a routing one: not several routes between fixed endpoints, but several occurrences of one component in one product.

8 Graphs: Cycles and the Scrap Loop

Finally, drop acyclicity: let G be an arbitrary finite directed graph. A cycle in G gives paths that loop, so in the directed container $\mathcal{N}_G$ the free category acquires nontrivial endomorphisms $v \to v$, and $P(v)$ becomes infinite—one path per number of laps (in the formalization, `Digraph.network G_scrap` in `Rungs.lean`). Nothing in Definition 4.2 minds: no finiteness and no acyclicity was ever assumed. That is the pure mathematics of this rung; the supply-chain reading is that a route may revisit a facility, and a tracking functor prices the laps like any other composite.

The steel network genuinely lives on this rung. Every mill produces *home scrap*—cobbles and offcuts from the rolling mill—which returns to the melt shop and is remelted. This is a leg rolling mill $\to$ melt shop closing a loop, and a tracking functor prices n remelt passes as the n-fold composite: yield loss multiplies, energy adds.

The BOM, however, remains a tree, and we can now say why rather than decree it: *scrap is a leaf of $\mathcal{B}$ because the parts list cuts the cycle.* Costing treats returning scrap as a purchased input, not as "made of rebar, which is made of billet, which is made of scrap, …"—the well-foundedness of the BOM is an accounting decision, and it is exactly what keeps explosion (comultiplication) terminating while the network loops freely.

One more inhabitant of this rung is still to come: the inventory container of Section 10, in which a receipt followed by an equal issue is already an endomorphism of the stock state.

Acyclicity, like thinness, is a property of examples, never a hypothesis of the theory.

9 Aggregation and Demand: Cofunctors

Functors are not the only structure-preserving maps between directed containers. A morphism of directed containers in the sense of Ahman–Uustalu maps shapes *forward* and positions *backward*—this is precisely a **cofunctor** (also called a retrofunctor) [2, 6].

Definition 9.1. A cofunctor $\Phi : \mathcal{C} \nrightarrow \mathcal{D}$ consists of a map $\varphi : \mathrm{Ob}\,\mathcal{C} \to \mathrm{Ob}\,\mathcal{D}$ together with, for each object c and each morphism $g : \varphi(c) \to d$ in $\mathcal{D}$, a chosen **lift** $\Phi_c(g) : c \to c_g$ in $\mathcal{C}$ with $\varphi(c_g) = d$, preserving identities and composition.

Construction 9.2. Let $\widetilde{\mathcal{N}} = \mathcal{N}_{\widetilde{G}}$ be the **aggregate network**—yet another application of Construction 5.1, this time to a two-node graph $\widetilde{G}$: shapes are regions or tiers (say *Australia* and *Thailand*, with a single leg from Australia to Thailand), positions are aggregate routes (in the formalization, `Digraph.network GAgg` in `Aggregation.lean`). A **routing policy** is a cofunctor $\Phi : \mathcal{N} \nrightarrow \widetilde{\mathcal{N}}$:

- on shapes, each facility is assigned its region: φ sends the vineyards and every Australian facility (through the Melbourne Port) to Australia, and the Thai Port, the distributors, and the retailers to Thailand;
- the lifting says: given a facility f and an *aggregate* route out of $\varphi(f)$—"move from Australia to Thailand"—the policy produces a *concrete* route out of f landing in the required region: from Sawasdee, the two legs Sawasdee → Melbourne Port → Thai Port; from Vineyard_1, the full route via Sawasdee. A single aggregate leg may lift to a many-leg concrete route—coarsening hides length, and the lift restores it.

The cofunctor laws say the policy is coherent: lifting an aggregate identity does nothing, and lifting a composite aggregate route agrees with lifting it in stages.

This is the mathematical shape of **demand fulfilment**. Demand is expressed at the aggregate level ("Thailand needs shiraz from Australia"); execution happens at the concrete level (specific facilities, specific routes). Planning systems are, structurally, machines for lifting coarse morphisms to fine ones—cofunctors.

Slogan 9.3. Goods flow along functors; demand flows along cofunctors. A functor pushes structure in the direction of its object map; a cofunctor pushes objects forward but pulls the instructions backward.

9.1 The two flows do not merge

It is tempting to look for a single category of "supply chain maps" containing both sourcing functors and routing cofunctors. The temptation should be resisted—and the literature agrees. Functors and cofunctors between fixed categories are genuinely different things, and neither subsumes the other. What exists instead is a **double category**: functors in one direction, cofunctors in the other, with squares expressing their compatibility. Clarke and Di Meglio study this double category of functors and retrofunctors [6]; Fairbanks, Carlson, and Spivak [7] recover it from the general theory of comonads, as the double category of polynomial comonads on **Set**.

For Sawasdee Cellars, a square in this double category relates: a tracking functor at the detailed level (duty, temperature, margin per leg, as in Section 5.1), the corresponding tracking at the aggregate level (total duty for the single leg Australia → Thailand), and the routing policies connecting the two levels—asserting that *aggregate compliance is consistent with per-leg compliance*: certifying the aggregate leg means certifying whichever concrete route the policy lifts it to. That such consistency is a square condition, not a commutation of two maps of the same kind, is the deepest structural fact this note has to offer: the disagreement between the two flows is preserved, not resolved.

Sourcing (Section 6.2) supplies a second family of squares of the same shape: a square relating detailed sourcing, aggregate sourcing ("rebar is made in the Emirates and sold in Europe"), and the routing policies connecting the two levels—aggregate planning consistent with detailed execution.

10 Inventory as an Update Monad

Directed containers whose position sets are constant—$P(s) = M$ for every shape s—are exactly **monoid actions**: M is a monoid (unit `o`, multiplication $\oplus$) acting on the set of shapes by $s \downarrow m$. This is a third construction of directed containers, distinct from both $\mathcal{N}_{(-)}$ and $\mathcal{B}$ (in the formalization, `MonoidAction.toDC` in `Inventory.lean`). Ahman and Uustalu [5] show these *cointerpret* as **update monads**

$$T(X) \;=\; \big(M \times X\big)^S,$$

whose Kleisli maps observe a state and emit an update.

The instance, at any stock-holding facility (a distributor's warehouse, say):

- shapes $S = \mathbb{N}^{\mathrm{Prod}}$ are **stock levels**;
- the monoid M is the free monoid of **stock movements** (receipts, issues, transfers), or its abelianization $\mathbb{Z}^{\mathrm{Prod}}$ if only net effect matters;
- the action $s \downarrow m$ applies the movements to the stock.

A **receipt** is stock arriving (goods inward); an **issue** is any authorized removal—issued *to production* (scrap and DRI consumed against a melt order, in the steel example of Section 6) or *for dispatch* (cases leaving for a retailer). The storekeeper's ledger has exactly two columns, and they are the generators of M. Note where these live in the directed container: a receipt is not a shape but a *position*—read as a category, a morphism $s \to s + r$ between stock states. The note's three directed containers render three different ontologies uniformly:

Directed container	Objects (shapes)	Morphisms (positions)
network $\mathcal{N}$	facilities	shipping routes
bills $\mathcal{B}$	BOM trees	component occurrences
inventory	stock levels	receipts, issues, transfers

On the ladder of Sections 5–8, this container sits on the graph rung: a receipt followed by an equal issue is a nontrivial endomorphism of the stock state.

A replenishment policy—look at current stock, decide what to order—is a Kleisli map for this monad. The same directed container, read comonadically, is the *costate* view: current stock together with the answer to every "what if these movements occur?" question. The monad/comonad pairing over one directed container captures the order-book/inventory duality: deciding updates versus anticipating them.

Remark 10.1. The choice between the free monoid and $\mathbb{Z}^{\mathrm{Prod}}$ is the choice between an *audit trail* and a *balance*: the abelianization quotient is exactly the information a ledger forgets.

Remark 10.2. Issues are *partial*: one cannot issue twelve bottles from a stock of five, and $\mathbb{N}^{\mathrm{Prod}}$ has no genuine subtraction. Totality of the action $s \downarrow m$ forces a modelling choice. Passing to $\mathbb{Z}^{\mathrm{Prod}}$ restores totality by admitting negative stock—which is not a fiction but a feature, namely *backorders*: demand accepted but not yet fulfillable. Truncated subtraction (monus) also keeps the action total but silently clamps at zero, discarding exactly the record of unmet demand. The algebraic choice is an operational policy: whether the system represents shortage or erases it.

Remark 10.3. A **transfer** is one physical event appearing in two directed containers at once: an issue at f, a shipping leg of $\mathcal{N}$ in transit, a receipt at g. Between issue and receipt the goods are *in transit*—stock that lives on a *morphism* of $\mathcal{N}$ rather than at an object. The statics of this section assign inventory only to facilities; tracking stock on routes is exactly where the statics start begging for the dynamics of Section 12.

11 Formalization in Lean

Every construction in this note has been mechanically verified in Lean 4 (no Mathlib, sorry-free), in the directed-containers library at `https://github.com/RaggedR/trust-boundaries` (directory `lean/`). The general theory—Definition 4.2 with laws D1–D5, the comonad structure on the extension and its converse, directed containers as small categories, and morphisms of directed containers as cofunctors—predates this note; the supply-chain constructions were added on top of it. The correspondence:

Paper item	Lean file	Verified statement
Def. 4.2	`Directed.lean`	directed containers, D1–D5
(§4)	`Comonoid.lean`, `ComonadConverse.lean`	DC structure ⇔ comonad
	`DContCat.lean`	DCs ⇔ small categories; cofunctors
Con. 5.1	`Network.lean`	any digraph yields a DC; D1/D2/D4 definitional; Sawasdee network instantiated
Prop. 5.2	`Thin.lean`	polytree ⇒ thin; $P(v) \cong$ reachable nodes
Con. 5.4	`Tracking.lean`	trackings $\cong$ edge labellings in M; max-temp, cold-chain, licence monoids
Con. 6.1, 6.3	`Sourcing.lean`, `DContFunctor.lean`	routed BOM trees as a DC; sourcing as a functor into $\mathcal{N}_{\text{steel}}^{\text{op}}$; Falcon Steel rebar instance
§7, §8	`Rungs.lean`	transshipment breaks thinness; the scrap loop makes $P(v)$ infinite
Con. 9.2	`Aggregation.lean`	Sawasdee routing policy satisfies the cofunctor laws
§10	`Inventory.lean`	monoid actions ⇔ constant-position DCs; update-monad laws; both Remarks

Two hypotheses were sharpened in passing from prose to proof. First, `Thin.lean` proves Proposition 5.2 from directed acyclicity plus edge-disjointness of co-sourced subtrees—a consequence of the undirected-acyclicity hypothesis stated in the proposition, and likely a slightly weaker one, so the mechanized theorem is if anything stronger. Second, Construction 6.3 as stated supplies sourcing data (a site per product, an inbound leg per BOM edge) for *all* of $\mathcal{B}$; but $\mathcal{B}$ contains shapes whose occurrences would require routes running backwards through the network, so no total assignment exists. The formalization instead equips each BOM shape with its routing choices (*routed* BOM trees), and sourcing becomes the canonical functor that reads them off; the data of Construction 6.3 builds exactly one routed BOM per plain BOM of interest.

12 Outlook: Dynamics in Poly

This note stayed within statics: networks, bills, assignments, policies. The same polynomial language extends to dynamics [11, 10]. Each facility becomes a Moore machine with interface $p_f = \sum_{\text{shipments}} y^{\text{orders}\times\text{deliveries}}$; the shipping graph becomes a wiring diagram; the whole chain becomes a single composite machine. Three phenomena then come into range:

- **Mode-dependence. Poly**'s distinguishing feature is that interfaces may change with the state. A supplier failure is not a bad input but a *change of interface*—the wiring diagram itself reconfigures. This is precisely the mode-dependent dynamics of [11].
- **The bullwhip effect.** Forrester's classical observation [8] that order variance amplifies upstream becomes a statement about the composite dynamical system—an invitation to study which wiring patterns and local policies produce it.
- **Plans running on infrastructure.** Libkind and Spivak's "pattern runs on matter" [9]—the free monad monad as a module over the cofree comonad comonad—is a general account of plans executing on machines. A replenishment order running on Sawasdee Cellars' network—or a works order on the steel mill of Section 6—is an instance begging to be worked out.

Each of these deserves its own note. Here we only observe that the statics were already comonadic, so the dynamics will not require a change of language—only a change of gear.

References

[1] M. Abbott, T. Altenkirch, N. Ghani. *Categories of containers.* FoSSaCS 2003.

[2] M. Aguiar. *Internal categories and quantum groups*. PhD thesis, Cornell University, 1997.

[3] D. Ahman, J. Chapman, T. Uustalu. *When is a container a comonad?* Logical Methods in Computer Science 10(3), 2014.

[4] D. Ahman, T. Uustalu. *Directed containers as categories*. MSFP 2016.

[5] D. Ahman, T. Uustalu. *Update monads: cointerpreting directed containers*. TYPES 2013.

[6] B. Clarke, M. Di Meglio. *An introduction to enriched cofunctors*. arXiv:2209.01144, 2022.

[7] A. D. Fairbanks, K. Carlson, D. I. Spivak. *Comonads as spaces*. arXiv:2607.15091, 2026.

[8] J. W. Forrester. *Industrial Dynamics*. MIT Press, 1961.

[9] S. Libkind, D. I. Spivak. *Pattern runs on matter: the free monad monad as a module over the cofree comonad comonad*. arXiv:2404.16321, 2024.

[10] N. Niu, D. I. Spivak. *Polynomial Functors: A Mathematical Theory of Interaction*. arXiv:2312.00990, 2023.

[11] D. I. Spivak. *Poly: an abundant categorical setting for mode-dependent dynamics*. arXiv:2005.01894, 2020.

[12] D. I. Spivak, R. Garner, A. D. Fairbanks. *Functorial aggregation*. arXiv:2111.10968, 2021.